ADR-0005 — Quelle stratégie de déploiement

Status : Accepté
Date : 09/10/2026 — révisé le 26/10/2026 (retour arrière par image GHCR)

Contexte

L'application est une seule API, qui tourne en un seul conteneur sur le VPS (ADR-0003). Chaque mise en production ne doit ni couper l'API ni perdre de commande, et une version défaillante doit pouvoir être retirée en quelques minutes.

Le serveur est partagé et limité à 8 Go de RAM : on ne peut pas y faire tourner plusieurs copies complètes de l'application en permanence.

Une version peut être défaillante de deux façons : elle ne démarre pas (modèle absent, stockage injoignable), ou elle démarre normalement mais se trompe dans ses réponses (bug métier). Le healthcheck ne détecte que la première.

Options considérées

Option	Avantage	Inconvénient
A. Big Bang Release	Simple : on arrête l'ancienne version, on démarre la nouvelle	Coupure à chaque mise en production, aucun filet en cas d'échec
B. Déploiement par service	Chaque service évolue à son rythme	Une seule API : rien à découper
C. Déploiement pilote	Nouvelle version testée par un groupe d'utilisateurs choisi	L'API ne connaît pas ses clients : pas de groupe à cibler
D. Canary Release	Nouvelle version exposée à une petite part du trafic	Plusieurs instances et un routage pondéré ; aucune métrique pour juger le canari avant `/metrics`
E. Blue / Green	Bascule instantanée et retour arrière immédiat	Deux environnements complets en permanence, trop lourd pour le VPS
F. Shadow	Nouveau modèle testé sur le trafic réel sans répondre aux clients	Deuxième instance et historique des prédictions pour comparer, pas encore disponibles
G. Rolling	Nouvelle version démarrée à côté de l'ancienne, trafic basculé seulement si elle est prête, natif dans Coolify	Les deux versions tournent ensemble quelques secondes

Décision

Option G : chaque mise en production est un rolling update de Coolify. Le nouveau conteneur démarre, Coolify attend que `/health/ready` réponde 200 (modèle chargé, stockage joignable), puis bascule le trafic et arrête l'ancien. Si la nouvelle version n'est jamais prête, l'ancienne continue de servir.

Coolify déploie l'image publiée par le pipeline sur GHCR (`ghcr.io/julcrm/ia-cloud-api`), désignée par son tag figé `sha-<commit>` plutôt que `latest`. Chaque version testée reste disponible dans le registre.

Retour arrière en cas de bug métier : on remet dans Coolify le tag `sha-<commit>` de la version précédente et on redéploie. C'est un rolling update comme un autre, vers une image déjà testée par le pipeline. Le rollback intégré de Coolify (images encore présentes sur le serveur) reste un secours.

Conséquences

`/health/ready` devient le healthcheck de Coolify, il ne doit répondre 200 que lorsque l'API peut vraiment traiter une requête.

Pendant la bascule, deux versions écrivent dans DuckLake en même temps, ce que le catalogue PostgreSQL supporte. Une nouvelle version doit donc rester compatible avec le schéma de l'ancienne.

Un retour arrière ne revient que sur le code : les commandes enregistrées restent, et l'ancienne image tourne avec les variables d'environnement actuelles. Schéma et configuration doivent rester compatibles avec la version précédente.

Les images GHCR ne doivent pas être supprimées tant qu'on peut vouloir y revenir.